

OntoSoft Process: Towards an agile process for ontology-based software

Joice B. Machado, Seiji Isotani,
Ellen F. Barbosa
University of São Paulo (USP/ICMC)
São Carlos (SP), Brazil
email: joicebm@icmc.usp.br,
sisotani@icmc.usp.br, francine@icmc.usp.br

Judson Bandeira, Williams Alcantara,
Armando Barbosa, Ig Bittencourt
University of Alagoas (UFAL)
Maceio (AL), Brazil
email: jmb@ic.ufal.br, wla@ic.ufal.br,
abs@ic.ufal.br, ig.ibert@ic.ufal.br

Abstract—Conceptual modeling, code automation and reduction of error propagation have motivated software development teams to use ontologies. The lack of methodologies and guidelines to associate ontologies and development software can interfere in the development time, as well as in the software quality. This paper proposes an agile process for ontology based software, called OntoSoft Process, bringing together practices from Software Engineering, Ontology Engineering and Scrum. Agile practices were chosen as they can be adapted according to the context of each application. The objective is to guide the development of ontology-based software by defining application scenarios and activities to develop the software and ontology. Furthermore, the aim is also to improve the collaboration and communication between software engineers and ontology engineers. OntoSoft has been applied to develop an ontology-based application to map and recommend real estates. Preliminary results suggest that the proposed process is effective in associating tasks, activities and in creating development scenarios.

Keywords—ontology engineering; software engineering; ontology-based software; agile development process; Scrum.

I. INTRODUCTION

Obtaining fewer errors in implementation and spending less time to deliver a stable product are two of the main factors related to the increasing use of ontologies in software development [21]. Ontologies can have a significant impact on the development of the components of an information system [11], bringing benefits such as automatic code generation and more accuracy when defining software requirements.

In short, an ontology is a formal specification of a conceptualization [10]. A conceptualization refers to the meaningfulness of concepts and their relationships given the context of the target world. A specification means a formal, declarative and explicit representation of them.

In Software Engineering (SE), conceptualization plays an important role throughout the software lifecycle. For instance, during analysis, modeling, design and development, all the relevant entities of the application domain, as well as their characteristics and relationships, should be conceptualized [1].

Ontologies have been considered appropriate instruments for communicating knowledge: (i) from project to project, in

a given target domain; and (ii) from one development phase to the next one, considering the software lifecycle [1, 13]. Thus, research on ontologies has emerged to investigate their effective use in software engineering.

Guarino (1998) [11] foresaw the benefits of adopting ontologies by introducing the Ontology-Driven Information System (ODIS) perspective, in which ontologies are used in the final application, without necessarily being adopted throughout the development process. This idea goes beyond Ontology Driven Software Engineering (ODSE), in which ontologies are used in the entire development process of the application [29].

Broadly speaking, the ODSE approach refers to how ontologies (and other semantic technologies) can contribute for improving Software Engineering both with respect to the process as well as to the produced artifacts [17].

Thus, many researchers adopt the Ontology Based Software Engineering (OBSE) term [1, 12, 13]. In this case, the ontology is the primary artifact and represents a central part of the application logic. The OBSE is particularly concerned with the application domain, such as the requirements analysis, domain modeling, integration, deployment, use and revision phases [13].

In this scenario, we have investigated effective processes (including tasks and related activities) to develop ontology-based software. In this paper, we define an agile process for ontology-based software, in which the ontology is used to support the system core, being considered one of the main software artifacts.

The current methodologies for software development are still not suitable to develop ontology-based systems [2]. By investigating these methodologies, we have identified some studies describing guidelines to develop ontology-based software. However, these guidelines are too generic, not dealing with ontology engineering (OE) and software engineering (SE) practices in order to outline the main tasks and roles of the developers.

Furthermore, the existing approaches are broad and their guidelines do not mention agile principles or practices to help in the iterative and incremental software development. This fact can make it difficult to do activities, considering

the developers' experience and knowledge. For untrained developers, these activities can consume too much time, and it is also complex to follow them throughout the project.

In our research, we have worked on setting up tasks and scenarios to help software development teams by considering agile management and practices of ontology engineering. We propose an agile process for ontology based software, called the OntoSoft (**Ontology based Software**) Process, by integrating SE and OE practices, as well as management activities from the Scrum agile method.

We used Scrum [23] to split up large tasks into small tasks in sprints in order to achieve results in every small delivery. Fragmenting large activities increases the possibility of identifying and managing the errors or inconsistencies that may arise during the development. Similarly to Scrum, the OntoSoft process is characterized by short development cycles and incremental planning to foster business needs.

The OntoSoft Process covers the stages of software and ontology development according to the team's background and knowledge, also considering the practices established in the Scrum. The guidelines aim to make the practices easily applied, feasible to be extensible and adaptable to the application context (industry or academia). Activities and tasks of the process occur in an iterative manner, contributing to the evaluation and evolution of the process, in order to: (i) prioritize customers' needs; (ii) optimize time; and (iii) enhance the team's skills.

To evaluate OntoSoft, a case study was planned and conducted. An ontology-based application, called Boa Moradia, was developed using the proposed process. The development team consisted of software engineers and ontology engineers, working collaboratively doing the activities in the process. Preliminary results show that the team of developers approve of the OntoSoft Process, and they claim the process has been effective in associating activities, tasks and defining application scenarios.

This paper is organized as follows. Section II presents a brief overview of development processes for ontology-based software. Section III describes the main characteristics of OntoSoft. Section IV describes the case study performed by applying OntoSoft and the preliminary results. Section V presents our conclusions and perspectives for future work.

II. RELATED WORK

Many approaches have considered ontologies in the conceptual stage of development, without however using them as artifacts that can enrich software implementation [24, 26, 28]. Usually, ontologies are used for extracting requirements and for modeling the domain, but they are not used for automating or optimizing implementation, for example [3].

The Kowien process model is a joint research project in the area of knowledge-based systems [7]. Kowien encourages reusing domain knowledge from specialist knowledge, especially in the initial phases of the project. However,

the association between ontology engineering and software engineering is not well specified and inference rules of the system are not detailed.

Hsieh and Lu [14] defined an ontology-based approach for collaborative software development. Ontologies are used in all phases of the development, but the process does not mention ontology engineering neither shows validation and specific rules for the defined phases.

The OBSE process (Ontology-Based Software Engineering) combines software design and ontology development cycles [1]. The ontology engineering phases are not specified, considering only the creation of ontologies in the initial phases of the project (requirements and analysis).

Deline et al. [5] proposed a development process for ontology-based software according to three levels: planning, architecture and specification of the ontologies. However, the process does not mention how to associate specific stages of software engineering nor ontology engineering.

Daramola [4] proposed a software process called PLOSATIS (Product Line for Ontology-based Semantics Aware Tourism Information Systems), which integrates software product lines, ontology engineering and semantic computing to develop tourism information systems. This approach defines the activities for requirements and analysis; ontology engineering, domain engineering and web product engineering.

Mavetera [19] proposed a generic approach for the ontology-based development, providing general guidelines for the main cycles of the software development process. However, the guidelines are still very broad to associate aspects of ontologies with software engineering.

Although these studies have important issues of ontology-based software development, only a few of them deal with the use of ontology engineering practices in software development. Regarding evaluation issues, most of the studies defined a framework or an application. However, these evaluations were restricted to a specific context, and the generalization of the process was not proposed for other scenarios.

Table I
SUMMARY OF RELATED WORK.

Approach	SE Activity	OE Activity	Ontology use in implementation
Kowien [7]	✓	✓	✓
Hsieh and Lu [14]	✓	✗	✗
OBSE [1]	✓	✗	✗
Deline et al. [5]	✓	✗	✗
Daramola [4]	✓	✓	✗
Mavetera [19]	✓	✗	✗
OntoSoft Process	✓	✓	✓

The main difference among OntoSoft and the approaches summarized herein consists of establishing roles, tasks and activities related to the development process. The granularity

of the tasks guide the developers' activities. Furthermore, the description of the scenarios and artifact examples in the process improved understanding in the process application. Such descriptions enable teams to adapt with different levels of experience when developing ontologies.

We have addressed ontology engineering tasks, integrating them with software activities. Different application scenarios have been considered, as well as the use of ontology as a software artifact. The integration activities between software and ontology are also described in detail and specified in each process activity, considering the agile practices of Scrum. The main characteristics of the OntoSoft Process are described next.

III. ONTOSOFT PROCESS: AN AGILE PROCESS FOR ONTOLOGY-BASED SOFTWARE

The OntoSoft Process has been proposed to create guidelines to develop ontology-based software. Our main motivation is to reduce the gap between Software Engineering (SE) and Ontology Engineering (OE) association by describing activities and tasks related to software and ontology during the development cycle.

OBSE has become an attractive paradigm in order to facilitate cooperation, to build compatible models, to reuse components and to achieve fewer costs in the software development [13]. In an ontology-based software, the ontology consists of a central artifact and is used to store and extract knowledge from the application. Furthermore, ontologies can provide enrichment to queries and automate development.

OntoSoft associates SE and OE practices, supported by the principles of the Scrum agile method, guiding the activities by sprint execution throughout the development. Ontology engineering facilitates the development of ontologies since it describes the steps to be followed by developers. In short, ontology engineering establishes activities to [6]: (i) organize knowledge acquisition; (ii) specify the primary goals, proposals, scope and granularity of ontologies; and (iii) build a vocabulary and organize an initial taxonomy, enabling the use of an intermediate representation of the requirements.

In the OntoSoft Process, Scrum activities, such as Daily Planning Meeting and Sprint Reviews can promote and encourage discussions among team members. These activities enable ongoing communication between ontology engineers and domain experts. Therefore, the changes are made continuously, preventing the spread of inconsistencies in the requirements. According to Scrum [25], a sprint is a time-box from 2 to 4 weeks, that generates a potentially releasable product increment (part of software or product).

Besides that, Scrum principles can ease the development of ontology-based software, due to the small and iterative cycle of activities. These characteristics contribute to decreasing the complexity of tasks, such as conceptualization

and knowledge acquisition, splitting them into smaller activities.

The main aspects of OntoSoft Process can be seen in Figure 1. The flow of activities starts with the ontology specification and requirements elicitation. The Product Backlog (the first artifact produced) is established in the first meeting, and consists of software and ontology items. Some sprints may contain activities related to the ontology and software, considering precedence and priority between tasks and artifacts.

At each Sprint Planning, the Sprint Backlog is defined and executed in accordance with the priorities established by the team. Daily Meetings are held in sub-teams: (i) meeting between ontology engineers and domain experts; and (ii) meeting between software engineers and application specialists. At the end of the sprint, in the Sprint Review, the sub-teams gather to identify probable inconsistencies, manage risks and design improvements.

This change in the daily meeting structure has become necessary because we realized that the initial activities related to ontology engineering (conceptualization and knowledge acquisition) require extensive and, sometimes, complex discussions. According to [13], ontology development is more long-term oriented and has a wider scope. Thus, when performing these activities we suggest having sub-teams in order not to hinder the progress of activities related to other software artifacts.

The OntoSoft Process defines tasks and roles for each activity. The roles can be categorized into: sponsor or customer, product owner, scrum master, software engineer, ontology engineer, application expert and domain expert. Stakeholders play different roles, either in sequential or in simultaneous activities.

OntoSoft consists of four stages: management, pre-development, development and post-development (Figure 2). Since the process is iterative, the activities can be related in different stages. For instance, ontology evaluation (development activity) can be started during knowledge acquisition (pre-development activity). As part of the OntoSoft Process, we provide the artifact template to make the developer's work easier.

Tasks related to ontology engineering activities are specified based on methodologies such as [8, 20, 27]. These methodologies were considered due to their similarities with the development processes in software engineering and due to their completeness as well. OntoSoft also uses concepts related to Ontology Design Patterns that favor reuse and good practices in the development of ontologies [22].

Finally, each activity of OntoSoft is associated with the creation or update of an artifact. Some of the artifacts generated are: sprint backlog, role definition, application scenarios, glossary, requirements document, ontology specification, diagrams (use case diagram, class diagram, sequence diagram, activities diagram), use case specification, ontology

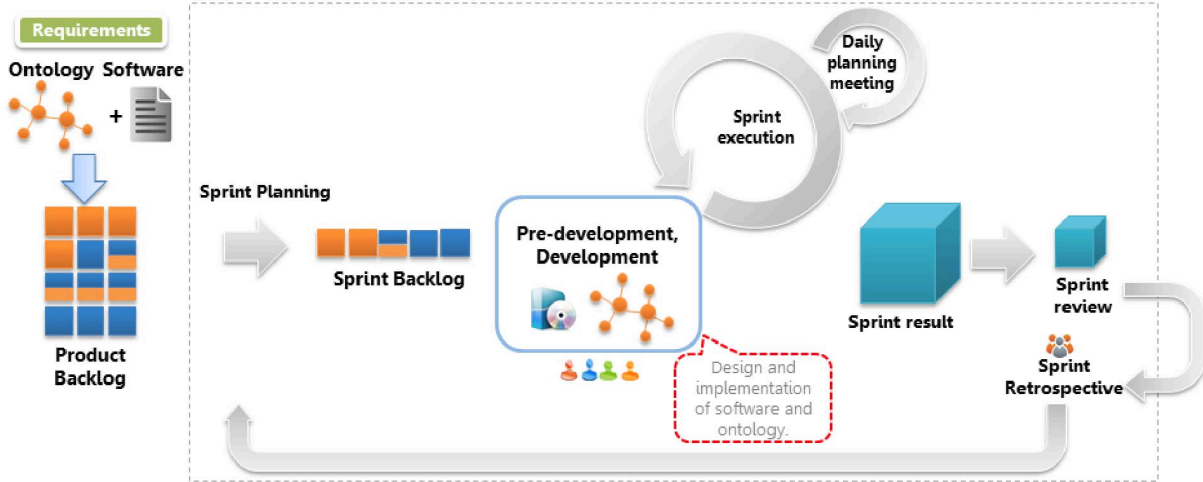


Figure 1. OntoSoft Process overview.

(RDF or OWL, for instance) and the system code.

The OntoSoft Process is suitable for three main scenarios of ontology-based software: **(i)** an ontology acting as the core (main software artifact) of a system; **(ii)** an ontology that is already developed and will be reused by a given system; and **(iii)** an ontology that should be initially developed to build a framework as a basis for other systems. These scenarios are discussed next.

A. OntoSoft Process: Application Scenarios

The OntoSoft Process behaves differently for each scenario, by establishing different flows of activities, according to the needs of the team.

Scenario I: The first scenario refers to the development of an application in which the core is the ontology. Figure 3 shows an overview of this scenario. In this case, the team chooses how many sprints will be necessary to specify the ontology before starting to extract the requirements.

If domain experts are part of the team and the idea is to start the ontology specification and the requirements extraction simultaneously, the tasks of OntoSoft can be executed in parallel. Considering this, the ontology engineer and software engineer roles should be well defined, so that the activities are not adversely affected or delayed. One activity can enhance the other and both should be consistent with each other. For instance, the conceptualization activity is directly related to the main requirements of the software.

Scenario II: The second scenario refers to the development of an application in which the ontology will be reused or imported into the application. Figure 4 shows this scenario overview. In this case, the process behaves similarly to the previous scenario.

OntoSoft Process uses some activities proposed by the third stage of NeOn methodology [18]. Ontology evaluation activities, selection and integration are applied to identify

ontological resources. The idea is to make the ontology to be reused in accordance with the application being developed. For instance, while verification and ontology reuse are carried out, the use of scenarios are also established in the same sprint.

In this case, we recommend developing or updating the terms of the glossary in order to check if the ontology is consistent with the goal of the application. Thus, changes may be made to avoid inconsistencies throughout the project and the ontology development.

Scenario III: In the third scenario, the ontology is used as the core of a platform that supports project development, playing an essential role in the composition of applications. In this case, in the platform definition, the ontology pre-development should be done iteratively and before the designing the application.

In the development of the semantic platform, the inclusion of new ontologies can generate new applications, as can be seen in Figure 5.

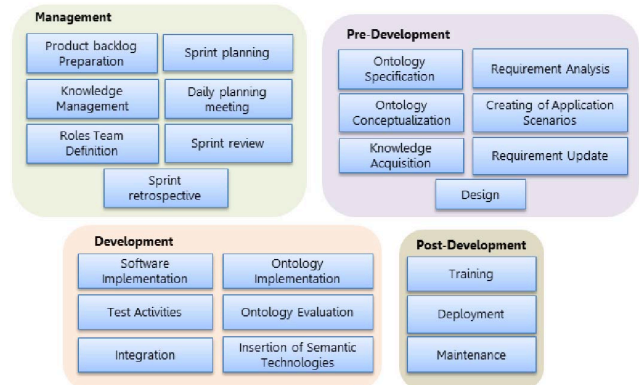


Figure 2. Activities of OntoSoft Process.



Figure 3. Scenario (i).

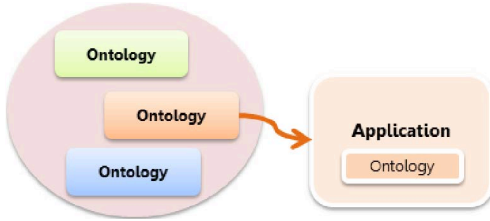


Figure 4. Scenario (ii).

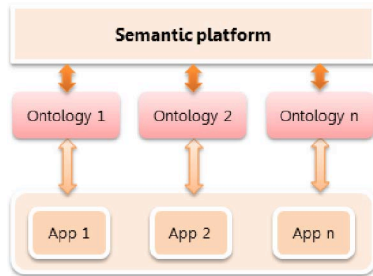


Figure 5. Scenario (iii).

Ontology specification, ontology conceptualization and knowledge acquisition should be iterated until the concept can be considered mature to initiate the software design. Moreover, these last two activities can contribute to the creation of application scenarios. Therefore, during the concept identification and formalization, software requirements can be identified simultaneously.

The third scenario can change when the platform is already finished. The platform is adapted and one or more ontologies can be inserted in the development of a new application. In this case, the activities of OntoSoft Process should be rearranged according to the needs of the team.

B. Description of OntoSoft Process Activities

The main stages of the OntoSoft Process are specified according to the goal and tasks to help the developers.

1) *Management*: This stage consists of activities that cover the software and the ontology development cycle, as well as its integration. Activities such as preparation of the product backlog, knowledge management and defining roles in the team are performed, as can be seen in Figure 6.

These activities occur in all the specified scenarios. Sprint Planning, Daily Planning Meeting, Sprint Review and Sprint



Figure 6. Management activities.

Retrospective are continuous activities in the sprint execution both for the ontology and for the software and integration. Table II presents the tasks related to management activities.

Table II
DESCRIPTION OF ACTIVITIES AND TASKS RELATED TO MANAGEMENT.

Activity	Tasks
Product Backlog Preparation	(i) Identify the main requirements and software functionality; (ii) associate priority to the identified item; (iii) associate the identified items to the software requirements.
Knowledge Management	(i) Identify the knowledge involved in the application; (ii) define how to map and enhance this knowledge through software; (iii) identify the probable semantic technologies that will be used.
Defining roles in the team	Define the roles: ontology engineer, software engineer, domain expert and manager (scrum master).
Sprint Planning	(i) Team Meeting with the sponsor; (ii) specify the customer needs; (iii) define the order of development sprints.
Daily Planning Meeting	(i) Identify what was done, what should be done and the problems found; (ii) manage the risks identified, for instance: technology changes, team performance, project complexity, among others.
Sprint Review	(i) Identify possible inconsistencies; (ii) adapt the activities; (iii) check the quality; (iv) manage future risks, based on the discussions raised.
Sprint Retrospective	(i) Discuss what has been done and what has not been done; (ii) evaluate the application (or the generated artifacts) consistency; (iii) observe the ontology and software development in parallel; (iv) improve the process continuously.

2) *Pre-development*: At this stage, the ontology specification and knowledge acquisition activities contribute to requirements extraction and to conceptual modeling. Activities related to the creation of application scenarios, used to define the application scope, are also conducted (Figure 7).

The process activities are performed according to the application scenario. For example, for scenario (i) the team can choose how many sprints will be required to specify the ontology before starting the requirements extraction. If the team has experts in the application domain and decides to start the ontology specification and the extraction requirements at the same time, the tasks of OntoSoft can be

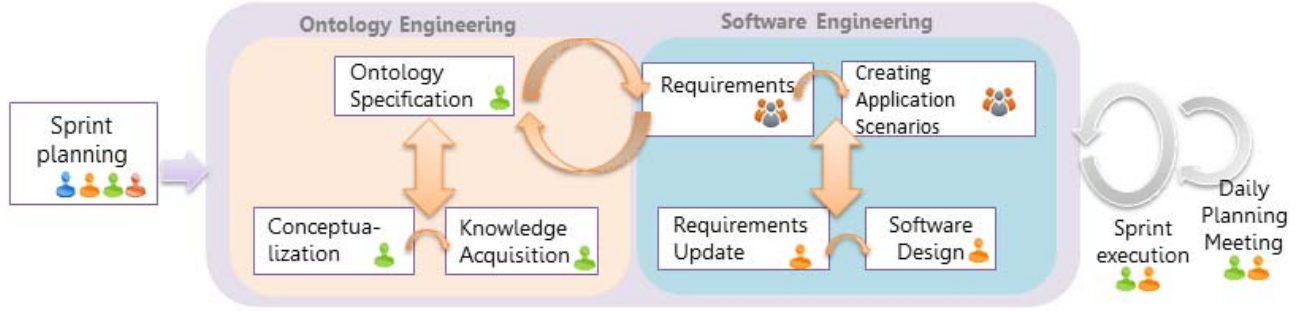


Figure 7. Pre-development activities.

performed simultaneously.

In the Daily Planning Meeting activity, the software engineer and the ontology engineer can talk about their doubts and decisions in order to reach a common understanding to develop the expected artifacts. Table III shows all the activities and tasks of pre-development.

3) *Development*: The implementation activities can occur in parallel, however the ontology should be finished firstly. In fact, some activities of software development depend on the finished ontology. Therefore, the Scrum Master should observe the dependency among activities, besides establishing their priority.

The ontology is evaluated by the team by applying the guidelines indicated in OntoSoft, and also according to the team's experience. These activities are updated and can modify the artifacts derived from the other sprints. Figure 8 shows an overview of the sequence of activities in the development stage.

The testing activities occur throughout the implementation activities in the ontology, software, and ontology and software integration. The evaluation activity occurs in a separate and integrated way. Based on the ontology formalization, the team can apply competency questions that help to evaluate the ontology concepts, during the pre-development. Thus, the evaluation activity starts in the pre-development and continues in the development.

The OntoSoft Process may also be specialized through the integration and use of semantic technologies. According to W3C, semantic web technologies can create data stores on the Web, build vocabularies and write rules for handling data. Technologies such as RDF, SPARQL, OWL, and SKOS are used¹. Semantic technologies comprise aspects of ontology engineering and management, as well as metadata creation, natural language processing and automated reasoning [9].

Furthermore, semantic technologies can be used to integrate the ontology and software in order to automate deployment and facilitate the integration and use of the ontology. They provide a flexible infrastructure for infor-

Table III
DESCRIPTION OF ACTIVITIES AND TASKS RELATED TO
PRE-DEVELOPMENT.

Activity	Tasks
Sprint Planning	(i) List the identified functionality; (ii) update the functionality and priorities previously set.
Ontology Specification	(i) Specify scope and ontology goal, raising the key concepts already identified; (ii) promote dialogue among ontology engineers to obtain common understanding of the domain.
Ontology Conceptualization	(i) Elicit the ontology concepts; (ii) search and query existing ontologies; (iii) consult generic ontologies; (iv) compare different ontologies versions; (v) attach to ontology design the information items with comments; (vi) consult the evaluation questions to verify the consistency of the identified concepts; (vii) store ontologies that will be used in a ontology library.
Knowledge acquisition	(i) Make the sketch of ontology; (ii) map the relationships between concepts; (iii) analyze the identified concepts considering the application scenarios suggested by the customer.
Requirements	(i) Investigate customer needs; (ii) identify the characteristics of application domain; (iii) identify and discuss the requirements; (iv) specify the goal and scope of the software and ontology.
Creation of Application Scenarios	Identify the requirements through scenarios, discussions, diagrams and storyboards in the project definition.
Update Requirements	Update the requirements considering the features extracted in the backlog items, vision document, application scenarios and glossary of terms.
Software Design	(i) Specify the use cases of the system; (ii) make diagrams required for the project; (iii) define the technologies to implement diagrams, ontology and to integrate ontology and software; (iv) choose the language and programming environment.

mation exchange, retrieval, sharing and data discovery [15]. Activities can be specified according to domain application and their definitions should start in the project management.

The development team and the Scrum Master participate

¹<http://www.w3.org/standards/semanticweb/>

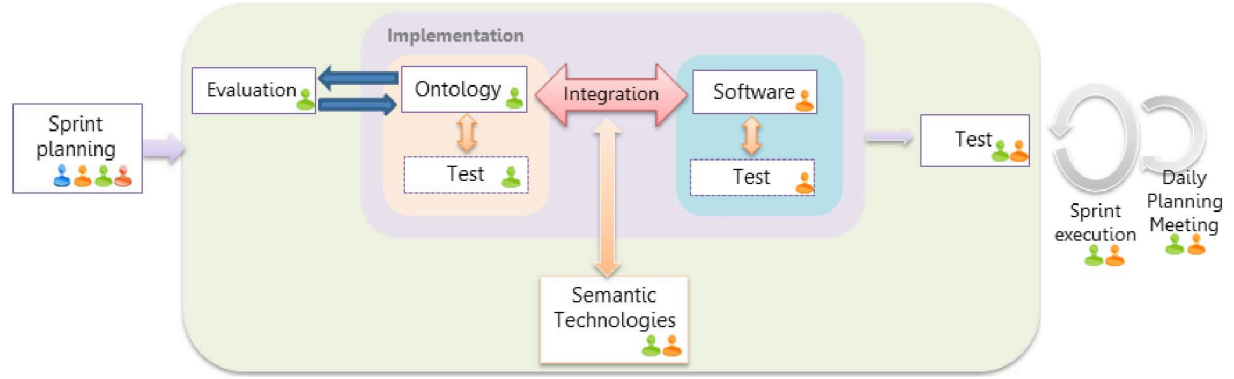


Figure 8. Development activities.

in the review and sprint retrospective, making necessary changes in each part of the software, in the ontology and in other related artifacts.

Table IV
DESCRIPTION OF ACTIVITIES AND DEVELOPMENT TASKS.

Activity	Tasks
Software Implementation	(i) Implement the software requirements and the models established in the previous activities in the project activity; (ii) use the ontology as a software component; (iii) choose a tool for ontology mapping.
Test	(i) Check the consistency of software and ontology; (ii) verify the correctness of the requirements and application scenarios; (iii) test the consistency of ontologies; (iv) unit testing; (v) integration testing.
Integration	(i) Establish priority and connection between the activities and their artifacts; (ii) define a tool to manipulate the ontology.
Ontology Implementation	(i) Select a tool to implement the ontology; (ii) formalize the specification; (iii) compare versions (if any); (iv) generalize versions; (v) manage the conceptualization.
Ontology Evaluation	Generate questions to assist the ontology evaluation, such as: clarity, coherence, extensibility, minimal bias in coding and ontological commitment.
Insertion of Semantic Technologies	Specify the activities related to the use or insertion of other semantic technologies.

4) *Post-Development*: In the Post-Development stage, the software is deployed and users are trained how to use it. Moreover, the maintenance activity helps to improve the software according to the customer's requirements. Thus, the software and ontology are updated, the product backlog is finalized and the product is delivered. Table V shows the post-development tasks and activities.

Table V
DESCRIPTION OF POST-DEVELOPMENT ACTIVITIES AND TASKS.

Activity	Tasks
Deployment	Software deployment.
Maintenance	Identify and fix the software and ontology inconsistencies.
Training	Training users.

IV. ONTOSOFT PROCESS USE: A CASE STUDY

According to Yin [16], a case study is used to deal with contextual conditions related to the phenomena of study. Since the OntoSoft process is targeted at a specific group of developers, namely teams with ontology engineers, we conducted a pilot case study to investigate the defined activities in a specific context. Our goal was to obtain feedback from experts and developers on ontologies (which in this case also includes software engineers) to improve and assess the proposed process. Table VI summarizes the objective, research question, hypothesis and case study propositions.

Table VI
CASE STUDY PROTOCOL

	Description
Objective	Evaluate the activities of OntoSoft Process and the team's adaptation to the process.
Research question	Does the process indicate the association between the software engineering and ontology engineering activities?
Hypothesis	1. The process activities are suitable for developing ontology-based software. 2. The proposed artifacts meet the team's needs.
Proposition	1. The management and communication between the team members are facilitated by Scrum activities.

Based on the case study protocol, we observed three sprint executions in the Boa Moradia System, an application to recommend real estates. Boa Moradia is a medium-sized

software, and corresponds to the first described scenario, in which ontology and software are developed simultaneously.

The system uses health data, safety data, education data, urban mobility and the human development index to produce information to qualify a city's neighbourhoods. To do so, the system uses open data, open government data, linked data and ontologies. Figure 9 presents part of the Boa Moradia ontology² developed by the team. For instance, the *Real Estate* concept represents any property, such as house, apartment or farm; and the *Offer* concept is related to a real estate that can be rented or sold.

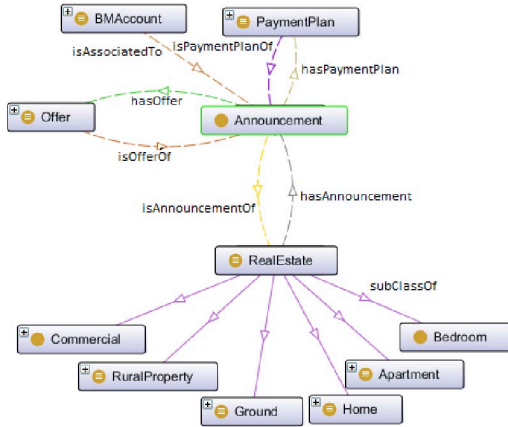


Figure 9. Boa Moradia Ontology.

The development team consists of three developers, computer science undergraduates and graduate students, with a background in ontology engineering and software engineering. The development of Boa Moradia was observed through weekly meetings with the development team and the appraiser of the process application. Collection and data monitoring were conducted using questionnaires, one for each finished sprint. We also used participant observation, interviews and physical artifacts to collect and analyze the data in order to investigate the research question.

Table VII presents some questions answered by the team. The first column shows the propositions of the case study. The second and third columns indicate the percentage of acceptance considered by developers, 25% to 50% and 75% to 100% respectively. On average, 66% of developers believe that the OntoSoft Process contributed to the development with an acceptance level above 75%. The association among Scrum values, software engineering and ontology engineering practices was considered 100% suitable for all developers.

This level of acceptance can be considered as evidence which supports the defined hypothesis in the protocol. According to the participants, the ontology engineering activities were included satisfactorily in the process. The

Table VII
EXAMPLE OF SOME QUESTIONS ANSWERED BY THE DEVELOPERS.

Proposition	Acceptance level between	
	25% and 50%	75% and 100%
OntoSoft Process met the team's needs.	33%	66%
Activities related to Scrum were embedded satisfactorily into the process.	–	100%
The tasks were enough to produce the artifacts.	33%	66%
The ontology engineering activities were included satisfactorily in the process.	–	100%

association among activities of ontology engineering and software engineering allowed for simultaneous development of the ontology and the software.

Despite being experts, developers also had difficulties with the application domain. This factor is very common in conceptualization and ontology specification activities, and this can interfere in the development time. However, by using OntoSoft, the team was able to define the activities and the priorities among them, avoiding late deliveries and fulfilling the deadlines.

Afterwards, in the third sprint, around 30 questions were asked. This questionnaire consists of questions related to all the activities of the process, such as design, ontology development, application development, as well as software and ontology integration.

In addition, some questions were taken from previous questionnaires in order to assess the OntoSoft progress, and the developer's commitment in following the guidelines of the process. Table VIII shows the fulfillment of activities and tasks of the process and Scrum practices.

Table VIII
SOME QUESTIONS ANSWERED BY THE DEVELOPERS.

Questions	Yes	No
Do the team members know about Scrum practices?	100%	–
Were Scrum practices followed by the team?	66%	33%
Were daily meetings held?	33%	66%
Were the artifacts developed following the OntoSoft Process?	100%	–
Were the tasks followed by the developers during the activities?	33%	66%

In the first sprints, some developers did not check the process before starting the development. This fact can be seen in Table VIII, as the tasks were often checked by only one of the developers and the Scrum practices (Daily Planning Meeting, for instance) were not followed. Thus, the Scrum Master and the appraiser of the process intervened and highlighted the need to follow the guidelines proposed.

²http://nees.com.br/linkn/onto/boa_moradia/

As well as these limitations, we identified some difficulties developers had regarding software design activities and insertion of semantic technologies. Based on the meetings, we tried to change and define the tasks, in a generic way, but indicating some steps to be followed by various types of applications. These activities are dependent on the context and change according to the application, and the tasks were not precisely outlined.

OntoSoft activities were discussed and improvements were made to refine the activities and tasks that were not consistent, such as the design activity and inserting semantic technologies. The flow of verification activities and preliminary assessment is shown in Figure 10.

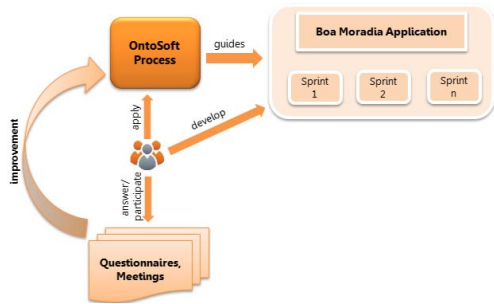


Figure 10. Assessment and improvement flow of OntoSoft Process.

The developers also emphasized the importance of: (i) identifying risks and adapting to teams (small, medium and large); and (ii) guiding the tasks related to semantic technology choice. These limitations have been considered and the process has been improved.

Activities and tasks related to implementing the ontology-based software were also considered in order to describe the relation among reasoners, code generation tools and ontology libraries. In fact, these guidelines are essential in helping the developer and reducing the development time. The idea is to contribute to the development through management and knowledge sharing. Furthermore, software engineers and ontology engineers can collaborate with each other and their skills may prevent possible difficulties of using new tools and/or deployment environments, for example exchanging experiences.

In general, according to the development team, one of the main contributions of the OntoSoft Process refers to the detail of activities and specified tasks. Defined tasks contribute to decision making and optimization of the development time.

V. CONCLUSIONS AND FUTURE WORK

In this paper, we presented OntoSoft, an agile process that associates software engineering practices and ontology engineering by setting guidelines to develop ontology-based software. The approach presented herein establishes the association between SE and OE, providing guidelines to

define activities, tasks, roles and artifacts throughout the development.

OntoSoft was applied in a pilot case study in the Boa Moradia software, which fits in the first scenario described in the process. The team had access to the detailed description of activities, tasks, roles and development artifacts. Therefore, the process has also been improved in some aspects considering the data analysis, such as the definition of details of some activities.

According to the evaluation performed, the activities are suitable for the context of Boa Moradia, demonstrating the flexibility of the OntoSoft process. In this application scenario, the development of software requirements and ontology occurred simultaneously. By using OntoSoft, the team could avoid errors of ambiguity, as well as reduce development time. Through Daily Meetings, Sprint Review and Sprint Retrospective, the process has provided a continuous update of artifacts.

Despite the benefits observed, when the developers do not follow the proposed guidelines, the accuracy of process cannot be measured with the adequate precision. A way to prevent this weakness is to conduct a training session on the process before the project starts. This practice is already being planned for the next experiments and case studies.

Moreover, during the sprint execution, we observed that the context of application interfered with the choice of the semantic technologies and this consumed development time. In order to solve this, in future work we intend to create a list of semantic technologies and tools, indicating their goal and their most appropriate uses. Such examples will be associated to the activities and tasks of the process.

By carrying out Scrum practices, the team's communication has also improved, especially in the case where the Sprint Reviews helped overcome some shortcomings and contributed to a better understanding of the application domain during the software development.

According to these results, some activities were better specified, contributing to the process improvement. To obtain more accurate statistics, the process will be evaluated again by other teams and applied in future experiments.

We also intend to describe the guideline specifications for the implementation activities and for investigating the integration of SE and OE tools and environments.

Currently, OntoSoft has been applied in the scenarios (ii) and (iii) by two different teams. Preliminary analyses and observations point out the advantages of using an agile approach to develop ontology-based applications.

ACKNOWLEDGMENTS

The authors acknowledge the Brazilian funding agencies (FAPESP, CAPES, CNPq) for their financial support and thank Dr. Timo Käkölä and the anonymous reviewers for their valuable comments and suggestions to improve the quality of the paper.

REFERENCES

- [1] A. Bachmann and W. Hesse. OBSE: an approach to Ontology-based Software Engineering in the practice. In *Proceedings of the 2nd International Workshop on Enterprise Modelling and Information Systems Architectures*, pages 129–142, 2007.
- [2] N. Baliyan and S. Kumar. Adaptation of software engineering to semantic web based system development. In *International Conference on Emerging Trends in Communication, Control, Signal Processing and Computing Applications, IEEE-C2SPCA 2013*, 2013.
- [3] M. V. Bossche and P. Ross. Ontology driven software engineering for real life applications. *3rd International Workshop on Semantic Web Enabled Software Engineering*, 2007.
- [4] O. J. Daramola. A process framework for semantics-aware tourism information systems. *Lecture Notes in Computer Science*, pages 521–532, 2010.
- [5] G. Deline, F. Lin, D. Wen, D. Gašević, and Kinshukb. A case study of ontology-driven development of intelligent educational systems. *International Journal of Web-Based Learning and Teaching Technologies*, 4:66–81, 2009.
- [6] V. Devedzić. Understanding ontological engineering. *Communications of the ACM*, 45(4):136–144, 2002.
- [7] L. Dittmann and S. Zelewski. Ontology-based skills management. In J.S Callaos, N. and Lefebvre, V. and Hansen, E and Dickopp, T. and Su, editor, *8th World Multi-Conference On Systemics, Cybernetics And Informatics*, pages 190–195, 2004.
- [8] M. Fernandez-Lopez, A. Gomez-Perez, and N. Juristo. Methontology: From ontological art towards ontological engineering. pages 33–40. Spring Symposium on Ontological Engineering of AAAI, 1997.
- [9] S. Grimm, A. Abecker, and J. Vo. In *Handbook of Semantic Web Technologies*. Berlin, Heidelberg.
- [10] T. R. Gruber. Towards principles for the design of ontologies used for knowledge sharing. *Int. Journal Human-Computer Studies*, (5/6), 1995.
- [11] N. Guarino. Formal Ontology and Information Systems. (June):443–456, 1998.
- [12] H. J Happel and S. Seedorf. Applications of ontologies in software engineering. In *International Workshop on Semantic Web Enabled Software Engineering*, pages 1–14, 2006.
- [13] W. Hesse. Ontologies in the Software Engineering process. *EAI 2005 - Proceedings of the Workshop on Enterprise Application Integration*, pages 3–16, 2005.
- [14] S.n Hsieh and M. Lu. In *Intelligent Computing in Engineering and Architecture*, pages 328–342.
- [15] S. John. Leveraging Traditional Software Engineering Tools to Ontology Engineering under a New Methodology. In *5th International Conference on Future Information Technology*, pages 1–5, 2010.
- [16] R. k. Yin. *Case Study Research: Design and Methods (Applied Social Research Methods)*, volume 5. Bookman, 2013.
- [17] A. Katasonov. Ontology-driven software engineering: beyond model checking and transformations. *Int. J. Semantic Computing*, 6(2):205–242, 2012.
- [18] Mari Carmen Suarez-Figueroa, Asuncion-Gomez-Perez and Mariano Fernandez-Lopez. *The NeOn Methodology for Ontology Engineering*. Berlin, Heidelberg, 2012.
- [19] N. Mavetera. Towards a Comprehensive Ontology-Driven Software Development Approach. In K.S. Soliman, editor, *Innovation and Knowledge Management: A Global Competitive Advantage*, pages 393–402. Int Business Informat Management Assoc, 2011.
- [20] A. Nicola, M. Missikoff, and R. Navigli. A proposal for a unified process for ontology building: UPON. In *Proceedings of the 16th International Conference on Database and Expert Systems Applications*, pages 655–664. Springer-Verlag, 2005.
- [21] J. Z. Pan, S. Staab, U. Abmann, J. Ebert, Y. Zhao, and D. Oberle. Ontology-driven software development. In *Ontology-Driven Software Development*, page 344. 2013.
- [22] V. Presutti, E. Daga, A. Gangemi, and E. Blomqvist. eXtreme design with content ontology design patterns. *CEUR Workshop Proceedings*, 516(ii):83–97, 2009.
- [23] K. S. Rubin. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. 2012.
- [24] J. Santhosh. Leveraging Traditional Software Engineering Tools to Ontology Engineering under a New Methodology. In *5th International Conference on Future Information Technology (FutureTech)*, pages 1–5, 2010.
- [25] K. Schwaber and J. Sutherland. The scrum guide. In *Scrum.org*, page 16, 2011.
- [26] K. Siegemund and E.J. Thomas. Towards ontology-driven requirements engineering. In *International Semantic Web Conference ISWC*, 2011.
- [27] M. C. Suárez-Figueroa. *NeOn Methodology for building ontology networks: specification, scheduling and reuse*. PhD thesis, 2010.
- [28] V. Tietz, A. Rumpel, M. Voigt, P. Siekmann, and K. Meissner. Tool support for semantic task modeling. In *Proceedings of the 3rd International Conference on Web Intelligence, Mining and Semantics*, pages 40:1 – 40:12, New York, 2013. ACM.
- [29] M. Uschold. Ontology-driven information systems: Past, present and future. *Frontiers in Artificial Intelligence and Applications*, 183(1):3–18, 2008.